

- Correction du TD 13 -

Algorithme de Dijkstra

Exercice 1 : Pour s'entraîner au tableau des poids minimaux

1. On considère le graphe ci-dessous :

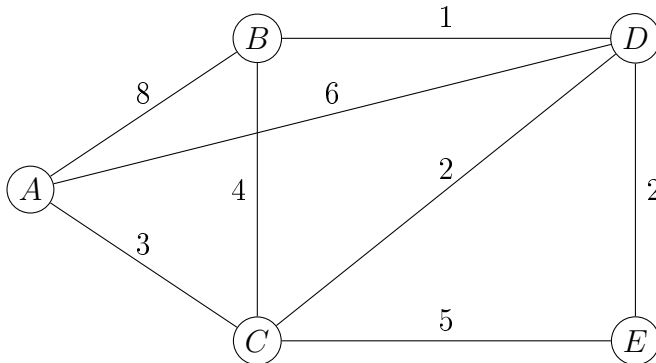
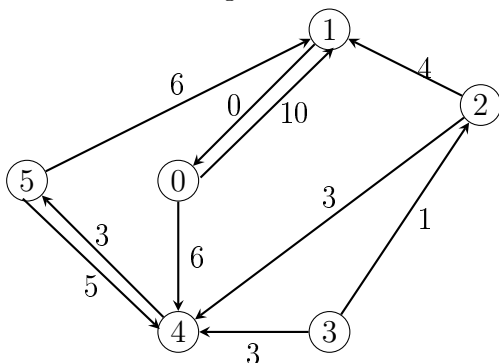


Tableau d'exploration des sommets obtenu à l'aide de l'algorithme de Dijkstra, pour obtenir la liste des poids des plus courts chemins issus du sommet A :

A	B	C	D	E
0	∞	∞	∞	∞
$\diagdown$	8 [A]	3 [A]	6 [A]	∞
$\diagdown$	7 [C]	$\diagdown$	5 [C]	8 [C]
$\diagdown$	6 [D]	$\diagdown$	$\diagdown$	7 [D]
$\diagdown$	$\diagdown$	$\diagdown$	$\diagdown$	7 [D]

2. On considère le graphe orienté et valué positivement suivant :

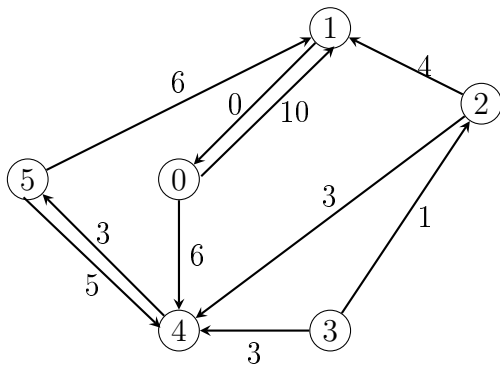


(a) Tableau d'exploration des sommets obtenu à l'aide de l'algorithme de Dijkstra, pour obtenir la liste des poids des plus courts chemins issus du sommet 3 :

0	1	2	3	4	5
∞	∞	∞	0	∞	∞
∞	∞	1 [3]		3 [3]	∞
∞	5 [2]			3 [3]	∞
∞	5 [2]				6 [4]
5 [1]					6 [4]
					6 [4]

D'après le tableau, un chemin de poids minimal reliant le sommet 3 au sommet 0 est (3, 2, 1, 0) et il est de poids 5.

- (b) Tableau d'exploration des sommets obtenu à l'aide de l'algorithme de Dijkstra, pour obtenir la liste des poids des plus courts chemins issus du sommet 0 :



0	1	2	3	4	5
0	∞	∞	∞	∞	∞
	10 [0]	∞	∞	6 [0]	∞
	10 [0]	∞	∞		9 [4]
	10 [0]	∞	∞		

A partir de cette étape, le tableau n'évolue plus.

En effet, seuls les sommets 1, 4 et 5 sont accessibles depuis le sommet 0.

Notons que le graphe orienté donné n'est pas fortement connexe.

Exercice 2 : Écriture de l'algorithme de Dijkstra

1. On remarque que `inf` est traité comme le symbole mathématiques $+\infty$.

2.

```
def initial(n,s):  
    Satraiter=list(range(n))  
    Satraiter.remove(s)  
    Poids=[float('inf')]*n  
    Poids[s]=0  
    Parent=[float('inf')]*n  
    Parent[s]='non'  
    return(Satraiter,Poids,Parent)
```

3.

```
def voisin(M,k,L):  
    u=[]  
    n=len(M)  
    for i in L:  
        if M[k,i]!=inf and k!=i:  
            u.append(i)  
    return(u)
```

4.

```
def echange(M,k,j,po,pa):  
    u=po[k]+M[k,j]  
    if u<po[j]:  
        po[j]=u  
        pa[j]=k  
    return(po,pa)
```

```

def Dijkstra(M,s):
    n=len(M)
    Satraiter,Poids,Parent=initial(n,s)
    k=s
    while Satraiter!=[]:
        V=voisin(M,k,Satraiter)
        for j in V:
            exchange(M,k,j,Poids,Parent)
        r=0
        j=Satraiter[r]
        smin=j
        m=Poids[smin]
        while r<len(Satraiter)-1:
            r=r+1
            j=Satraiter[r]
            if Poids[j]<m:
                smin=j
                m=Poids[j]
        Satraiter.remove(smin)
        k=smin
    return(Poids,Parent)

```

6. (a) La matrice M des poids des arêtes associée au graphe donné est :

$$M = \begin{pmatrix} \infty & 3 & 1 & \infty & \infty & \infty \\ 3 & \infty & 1 & 2 & \infty & \infty \\ 1 & 1 & \infty & 3 & 5 & \infty \\ \infty & 2 & 3 & \infty & 1 & 3 \\ \infty & \infty & 5 & 1 & \infty & 1 \\ \infty & \infty & \infty & 3 & 1 & \infty \end{pmatrix}$$

En Python, on écrit par exemple :

```

M=np.array([[inf,3,1,inf,inf,inf],\
            [3,inf,1,2,inf,inf],\
            [1,1,inf,3,5,inf],\
            [inf,2,3,inf,1,3],\
            [inf,inf,5,1,inf,1],\
            [inf,inf,inf,3,1,inf]])

Po,Pa=Dijkstra(M,0)

print('Poids='+repr(Po))
print('Parent='+repr(Pa))

```

(b) La matrice M des poids des arêtes associée au graphe est :

$$M = \begin{pmatrix} \infty & 1 & 1 & 1 & \infty & \infty & \infty \\ 1 & \infty & \infty & \infty & 2 & 2 & \infty \\ 1 & \infty & \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & 2 & \infty & \infty & \infty & \infty & 3 \\ \infty & 2 & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 3 & \infty & \infty \end{pmatrix}$$

En Python, on écrit par exemple :

```
M=np.array([[inf,1,1,1,inf,inf,inf],\
            [1,inf,inf,inf,2,2,inf],\
            [1,inf,inf,inf,inf,inf,inf],\
            [1,inf,inf,inf,inf,inf,inf],\
            [inf,2,inf,inf,inf,inf,3],\
            [inf,2,inf,inf,inf,inf,inf],\
            [inf,inf,inf,inf,3,inf,inf]])

Po,Pa=Dijkstra(M,0)

print('Poids='+repr(Po))
print('Parent='+repr(Pa))
```

7. L'algorithme écrit dans la fonction `Dijkstra` s'arrête lorsque la liste `Satrait` est vide. A chaque passage dans la boucle `while Satrait!=[]`, même si le sommet traité (noté `k`) n'a plus de voisins (ou successeurs) qui n'ont pas encore été traités (c'est-à-dire si le tableau que l'on complète à la main, comme dans l'exercice 1, n'évolue plus), l'algorithme cherchera parmi les sommets j pour lesquels $\delta(s, j)$ n'a pas encore été trouvé, celui de valeur minimale (éventuellement égale à ∞), à enlever le sommet j trouvé de la liste `Satrait`, avant de reprendre la boucle `while` du début.

En résumé, que l'on puisse se déplacer ou non à partir du sommet `k` où l'on est placé lors d'un passage dans la boucle, à la fin de la boucle `while`, on a enlevé un des sommets de la liste `Satrait`. Donc à l'issue du $(n - 1)$ ^{ième} passage dans la boucle `while`, la liste `Satrait` est vide, et l'algorithme s'arrête.

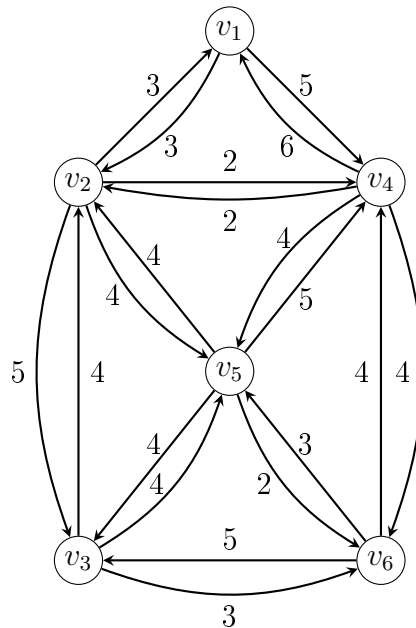
8

```
def chemin(M,i,j):
    po,pa=Dijkstra(M,i)
    k=j
    u=[k]
    while k!=i:
        m=pa[k]
        u.insert(0,m)
        k=m
    return(u)
```

Exercice 3 : Une première application

1. Traçons le graphe associé aux durées de vols :

- les sommets du graphe sont les six villes v_1, v_2, v_3, v_4, v_5 et v_6 ,
- la matrices M n'étant pas symétrique, le graphe est orienté,
- deux sommets v_i et v_j , avec $i \neq j$, sont reliés par un arc pondéré $\overset{d_{i,j}}{\curvearrowright} v_i \rightarrow v_j$ si il existe un avion reliant la ville v_i à la ville v_j , le poids de l'arc $d_{i,j}$ indiquant la durée du vol
- une absence d'arc entre deux villes indique qu'il n'existe pas de vol reliant ces deux villes.

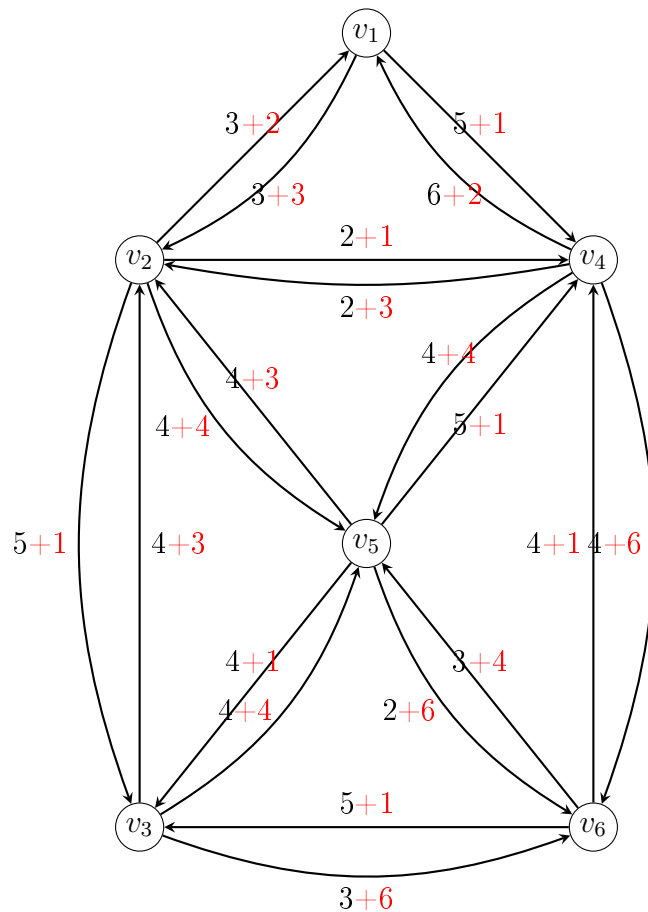


Complétons le tableau de l'algorithme de Dijkstra pour trouver le chemin de poids minimal reliant la ville v_1 à la ville v_6 .

v_1	v_2	v_3	v_4	v_5	v_6
0	∞	∞	∞	∞	∞
	3 $[v_1]$	∞	5 $[v_1]$	∞	∞
		8 $[v_2]$	5 $[v_1]$	7 $[v_2]$	∞
		8 $[v_2]$		7 $[v_2]$	9 $[v_4]$
		8 $[v_2]$			9 $[v_4]$
					9 $[v_4]$

On en déduit que le trajet le plus rapide pour relier les villes v_1 à v_6 dure 9 heures, et un tel trajet peut être celui passant successivement par les villes : v_1, v_4, v_6 .

2. On ajoute les temps d'escale aux temps de trajets :



On adapte le tableau précédent en ajoutant les temps d'escale (en rouge) dès que l'on change d'avion. Le nouveau tableau est donc le suivant :

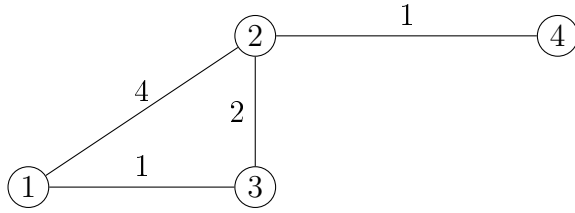
v_1	v_2	v_3	v_4	v_5	v_6
0	∞	∞	∞	∞	∞
	$3+3 [v_1]$	∞	$5+1 [v_1]$	∞	∞
		$6+5+1 [v_2]$	$5+1 [v_1]$	$6+4+4 [v_2]$	∞
		$11+1 [v_2]$		$10+4 [v_2]$	$6+4+6 [v_4]$
		$11 [v_2]$			$10 [v_4]$
		$11 [v_2]$			

Avec les temps d'escale, le trajet $v_1 - v_4 - v_6$ est toujours le plus rapide, mais il met 10 h dorénavant.

Exercice 4 : L'algorithme de Maria Hasse

1. Étude d'un exemple

On considère le graphe $G = (S, A, \nu)$, où $S = \{1, 2, 3, 4\}$, valué ci-dessous :



(a) On a :

$$M = \begin{pmatrix} 0 & 4 & 1 & \infty \\ 4 & 0 & 2 & 1 \\ 1 & 2 & 0 & \infty \\ \infty & 1 & \infty & 0 \end{pmatrix}$$

(b) Pour le calcul de $A \otimes B$: pour chaque coefficient de la matrice $A \otimes B$, on travaille comme pour le produit matriciel usuel.

Pour calculer le coefficient situé en ligne i et colonne j de $A \otimes B$:

→ on balaye la ligne n° i de A et la colonne n° j de B ,

→ on additionne les deux coefficients correspondants (le terme de la $i^{\text{ième}}$ ligne de A additionné au terme de la $j^{\text{ième}}$ colonne de B) : on obtient n nombres,

→ on conserve la plus petite valeur parmi ces n nombres.

On obtient ainsi :

$$M^{\otimes 2} = M \otimes M = \begin{pmatrix} 0 & 4 & 1 & \infty \\ 4 & 0 & 2 & 1 \\ 1 & 2 & 0 & \infty \\ \infty & 1 & \infty & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & 4 & 1 & \infty \\ 4 & 0 & 2 & 1 \\ 1 & 2 & 0 & \infty \\ \infty & 1 & \infty & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 1 & 5 \\ 3 & 0 & 2 & 1 \\ 1 & 2 & 0 & 3 \\ 5 & 1 & 3 & 0 \end{pmatrix}$$

puis :

$$M^{\otimes 3} = M^{\otimes 2} \otimes M = \begin{pmatrix} 0 & 3 & 1 & 5 \\ 3 & 0 & 2 & 1 \\ 1 & 2 & 0 & 3 \\ 5 & 1 & 3 & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & 4 & 1 & \infty \\ 4 & 0 & 2 & 1 \\ 1 & 2 & 0 & \infty \\ \infty & 1 & \infty & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 1 & 4 \\ 3 & 0 & 2 & 1 \\ 1 & 2 & 0 & 3 \\ 4 & 1 & 3 & 0 \end{pmatrix}$$

et enfin :

$$\begin{aligned} M^{\otimes 4} &= M^{\otimes 3} \otimes M = \begin{pmatrix} 0 & 3 & 1 & 4 \\ 3 & 0 & 2 & 1 \\ 1 & 2 & 0 & 3 \\ 4 & 1 & 3 & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & 4 & 1 & \infty \\ 4 & 0 & 2 & 1 \\ 1 & 2 & 0 & \infty \\ \infty & 1 & \infty & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 1 & 4 \\ 3 & 0 & 2 & 1 \\ 1 & 2 & 0 & 3 \\ 4 & 1 & 3 & 0 \end{pmatrix} \\ &= M^{\otimes 3} \end{aligned}$$

(c) En observant le graphique, on obtient directement :

Pour $s = 1$

Sommet :	1	2	3	4
Poids :	0	3	1	4

Pour $s = 2$

Sommet :	1	2	3	4
Poids :	3	0	2	1

Pour $s = 3$

Sommet :	1	2	3	4
Poids :	1	2	0	3

Pour $s = 4$

Sommet :	1	2	3	4
Poids :	4	1	3	0

Les quatre listes des poids minimaux sont exactement les quatre lignes de la matrice $M^{\otimes 3}$.

2. Soit $G = (S, A, \nu)$ un graphe d'ordre n , valué positivement, et ne possédant pas de boucle. On notera $1, 2, \dots, n$ les sommets de G .

On note M la matrice de $\mathcal{M}_n(\mathbb{R} \cup \{\infty\})$ donnant le poids des arêtes (ou arcs) comme dans l'exemple de la question précédente, une absence d'arête (ou d'arc) entre deux sommets étant signalée par ∞ .

- (a) Pour $k = 1$, par définition de la matrice $M^{\otimes 1} = M$, on a bien la description donnée dans l'énoncé.

Soit $k \in \mathbb{N}^*$ tel que, pour tout $(i, j) \in \llbracket 1, n \rrbracket^2$, le coefficient situé à la $i^{\text{ième}}$ ligne et $j^{\text{ième}}$ colonne est égal à :

- 0 si $i = j$,
- ∞ si il n'existe pas de chemin (ou chaîne) de longueur $\leq k$ et reliant le sommet i au sommet $j \neq i$ de G ,
- le poids du chemin de poids minimal et de longueur $\leq k$ et reliant le sommet i au sommet $j \neq i$ de G .

Par définition, $M^{\otimes(k+1)} = M^{\otimes k} \times M$.

$$\text{On note : } \begin{cases} M^{\otimes(k+1)} = (c_{i,j})_{1 \leq i,j \leq n} \\ M^{\otimes k} = (a_{i,j})_{1 \leq i,j \leq n} \\ M = (m_{i,j})_{1 \leq i,j \leq n} \end{cases}$$

Soit $(i, j) \in \llbracket 1, n \rrbracket^2$.

Par définition : $c_{i,j} = \min_{1 \leq \ell \leq n} (a_{i,\ell} + m_{\ell,j})$.

Cas $i = j$

Par définition de M et par hypothèse de récurrence, pour tout $\ell \in \llbracket 1, n \rrbracket$,

$a_{i,\ell} \in \mathbb{R}_+ \cup \{\infty\}$ et $m_{\ell,i} \in \mathbb{R}_+ \cup \{\infty\}$.

Donc, pour tout $\ell \in \llbracket 1, n \rrbracket$, $a_{i,\ell} + m_{\ell,i} \geq 0 = a_{i,i} + m_{i,i}$.

D'où $c_{i,i} = 0$.

Cas $i \neq j$

Supposons que i et j sont reliés par un chemin de longueur $\leq k + 1$

Pour construire un chemin de longueur $\leq k + 1$ reliant i à j , il faut et il suffit de relier un voisin (ou prédécesseur) ℓ de j à i par une chaîne (ou chemin) de longueur $\leq k$.

Pour déterminer le chemin de poids minimal et de longueur $\leq k + 1$ reliant i à j , on doit prendre le chemin de poids minimal et de longueur $\leq k$ reliant i à ℓ , où ℓ est un voisin de j . Parmi tous ces chemins possibles, celui de poids minimal a pour poids :

$$\min_{1 \leq \ell \leq n} (a_{i,\ell} + m_{\ell,j}).$$

Donc $c_{i,j}$ donne bien le poids du chemin de longueur $\leq k + 1$, de poids minimal et reliant i à j .

Supposons que i et j ne sont pas reliés par un chemin de longueur $\leq k + 1$

Comme ci-dessus, comme il n'existe pas de chemin de longueur $\leq k + 1$ reliant i à j , cela signifie que pour tout voisin ℓ de j , il n'existe pas de chemin de longueur $\leq k$ reliant i à ℓ .

$$\text{Donc, dans ce cas : } \forall \ell \in \llbracket 1, n \rrbracket, a_{i,\ell} + m_{\ell,j} = \begin{cases} \infty + m_{\ell,j} \\ \text{ou} \\ a_{i,\ell} + \infty \end{cases} = \infty$$

D'où $c_{i,j} = \infty$. Et ∞ indique bien que les sommets i et j ne sont pas reliés par un chemin de longueur $\leq k + 1$.

La propriété est donc héréditaire.

On déduit du principe de récurrence que, pour tout $k \in \mathbb{N}^*$, la matrice $M^{\otimes k}$ donne bien la liste des poids des chemins minimaux de longueur $\leq k + 1$.

(b) Soit $k \in \mathbb{N}^*$.

Soit $(i, j) \in \llbracket 1, n \rrbracket^2$.

Par définition : $m_{i,j}^{k+1} = \min_{1 \leq \ell \leq n} (m_{i,\ell}^{\otimes k} + m_{\ell,j}) \leq m_{i,j}^{\otimes k} + \underbrace{m_{j,j}}_{=0} = m_{i,j}^{\otimes k}$.

(c) Supposons qu'il existe $k \in \mathbb{N}^*$ tel que : $M^{\otimes k} = M^{\otimes(k+1)}$.

Par récurrence, on obtient alors que : $\forall p \geq k, M^{\otimes p} = M^{\otimes k}$.

On note encore : $M^{\otimes k} = (m_{i,j}^{\otimes k})_{1 \leq i,j \leq n}$ et $M^{\otimes(k+1)} = (m_{i,j}^{\otimes(k+1)})_{1 \leq i,j \leq n}$.

Soit $(i, j) \in \llbracket 1, n \rrbracket^2$.

Cas $i = j$

Par convention, le chemin de poids minimal reliant i à i a pour poids 0.

Et d'après la question a, le coefficient situé à la $i^{\text{ième}}$ ligne et $i^{\text{ième}}$ colonne de $M^{\otimes k}$ est égale à 0.

Cas $i \neq j$ et $m_{i,j}^{\otimes k} \in \mathbb{R}$

Pour tout $p \geq k, m_{i,j}^{\otimes p} = m_{i,j}^{\otimes k}$.

Donc d'après a, pour tout $p \geq k$, le chemin de poids minimal de longueur $\leq p$ et reliant i à j a le même poids que le chemin de poids minimal, de longueur $\leq k$ et reliant i à j . Il n'est plus utile de chercher d'autres chemins reliant i à j : le chemin trouvé de poids minimal et de longueur $\leq k$ convient.

Donc : $\delta(i, j) = m_{i,j}^{\otimes k}$.

Cas $i \neq j$ et $m_{i,j}^{\otimes k} = \infty$

Comme $m_{i,j}^{\otimes k} = \infty$, il n'existe pas de chemins de longueur $\leq k$ reliant i à j .

Comme pour tout $p \geq k, m_{i,j}^{\otimes p} = \infty$, on a de même : il n'existe pas de chemin reliant i à j .

D'où : $\delta(i, j) = \infty = m_{i,j}^{\otimes k}$.

(d) Pour déterminer un chemin de poids minimal reliant deux sommets du graphe, le chemin est nécessairement élémentaire (car le graphe est valué positivement).

Donc les chemins de poids minimaux sont nécessairement de longueur $\leq n - 1$.

Donc, pour $k = n - 1$, la matrice $M^{\otimes(n-1)}$ donne la liste des poids minimaux des chemins de poids minimal reliant chaque couple de sommets. Et on en déduit que nécessairement : $M^{\otimes n} = M^{\otimes(n-1)}$.

3. Application Python

(a) Un exemple de fonction Prod :

```
def Prod(A, B):
    (p, q) = np.shape(A)
    M = np.zeros([p, q])
    for i in range(p):
        for j in range(q):
            M[i, j] = np.min(A[i, :] + B[:, j])
    return M
```

(b) Un exemple de fonction Poids :

```
def Poids(M,i,j):  
    A=M  
    B=Prod(A,A)  
    while np.allclose(A,B)==False:  
        A=B  
        B=Prod(B,M)  
    return(A[i,j])
```
